

INSTITUTO FEDERAL DO PIAUÍ

Curso de Superior de Tecnologia em Análise e Desenvolvimento de Sistemas
Programação para Internet II, Prof. Rogério Silva, ADS IV – 2026.2

Atividade G - Arquiteturas em Camadas, ORM e Autenticação

Modalidade: dupla ou individualmente

Projeto: base-`mini-prontuario-t3`

Prazo: definido no Classroom

Objetivo: você vai transformar a arquitetura de um sistema que funciona — sem quebrá-lo — e depois vai trancá-lo com identidade e permissões. A prova de que você conseguiu não é a sua palavra: é o `npm run gate` verde e os testes de ataque passando.

1. O contexto

O Mini-Prontuário chega até você **funcionando de ponta a ponta**: pacientes, atendimentos, prescrições, upload de foto, validação, erros bem-formados. O frontend está pronto (e é **intocável**) — inclusive a tela de login e o crachá de papel do usuário, que hoje dormem e **acendem sozinhos** quando o seu backend de identidade nascer.

O que esta atividade pede não é "fazer mais features". É o trabalho que distingue um dev Pleno/Sênior: **mudar a estrutura por baixo de um sistema em uso** (trilhas ARQ e ORM) e **colocar segurança no lugar certo da arquitetura** (trilha AUTH). Tudo isso conduzindo IA — chat ou agente — com método, porque é assim que esse trabalho acontece em 2026.

O seu papel (e é nisso que você será avaliado): compreender o problema, **especificar** (o quê, por quê, restrições, estado de pronto), e **aceitar ou recusar** o que a IA produzir — com testes e invariantes, não com impressão.

2. O que já existe × o que falta

Já existe (não reescreva)	Falta (o seu trabalho)
Camadas Route/Controller/Service completas para Patient, Encounter e MedicationRequest	Camada Repository (interface + adapter) — trilha ARQ
<code>HttpError</code> + <code>errorHandler</code> (contrato de erro), Zod, upload Multer	<code>UnauthorizedError/ForbiddenError</code> e todo o backend de identidade — trilha AUTH
Banco SQLite com <code>better-sqlite3</code> , schema + seed	Prisma atrás da mesma interface + migrations — trilha ORM

Já existe (não reescreva)	Falta (o seu trabalho)
<code>gate.sh</code> , <code>.dependency-cruiser.cjs</code> , testes de fumaça, testes de ataque (dormindo)	Deixar o gate inteiro verde — inclusive os ataques
Fundação de IA: <code>AGENTS.md</code> , <code>CLAUDE.md</code> , <code>INVARIANTES.md</code> , <code>docs/code_review.md</code>	O seu <code>IA.md</code> (template na Seção 6) e, no N3, uma ADR
Frontend completo com login/papéis prontos	Nada — é proibido tocá-lo

O mapa detalhado dos TODOs (ARQ-1..6, ORM-1..5, AUTH-1..8) está no **README.md do projeto** e nos comentários do código. A Apresentação de Condução acompanha as trilhas passo a passo — **até certo ponto** (ver N2).

3. Regras de escopo (leia duas vezes)

1. **Bibliotecas permitidas** — e nada além delas, para você e para a sua IA: `express` · `better-sqlite3` · `zod` · `multer` · `prisma/@prisma/client` · `argon2` · `jsonwebtoken` · `dotenv`.
2. **Frontend congelado**. Nenhum arquivo de `public/` muda. Se o seu backend estiver certo, a tela de login acende sozinha.
3. `tests/api.smoke.test.ts` é **intocável** nas trilhas ARQ e ORM — ele é a definição executável de "refatorar **sem quebrar**". Se um teste "atrapalha", o problema está no seu código.
4. `gate.sh` e `.dependency-cruiser.cjs` **não são editáveis para "passar"**. A única mudança permitida na régua é o TODO ARQ-6 (que **adiciona** uma regra). Gate desabilitado ou afrouxado = entrega reprovada, em qualquer nível.
5. **Segredos só no `.env`** (que nunca vai para o repositório). O `JWT_SECRET` no código é reprovação do critério de segurança.
6. **Migrations são código versionado**. A partir da baseline do Prisma, nenhuma mudança de esquema fora de `prisma migrate dev` (invariante OP-1).

4. Protocolo de IA — as duas rotas (obrigatório)

A IA é **incentivada** — vibe coding é **proibido**. A diferença tem nome: *conduzir* ≠ *vibrar*. Você pode trabalhar por qualquer combinação das duas rotas abaixo; ambas são auditáveis pelo seu `IA.md`.

Rota chat (Claude/ChatGPT/Gemini no navegador)

Como desde o Tópico 1: prompts no formato **C-P-T-R-F-A** e validação **V.E.R.M.** antes de qualquer colagem no projeto. Nada de colar bloco que você não consegue explicar linha a linha.

Rota agente (Claude Code · Codex · Antigravity CLI **agy**)

Formalmente liberada neste tópico, **sob o método do ebook da disciplina**:

1. **Toda tarefa escrita nos 4 campos** — *Goal / Context / Constraints / Done when*. O **Done when** precisa ser algo que **uma máquina avalia** (ex.: **npm run gate verde**; "ATAQUE 3 do auth.attacks passa"). "Funcionar direitinho" não é Done when.
2. **Plan Mode obrigatório em tarefa multi-arquivo** (toda a trilha ARQ e a ORM são multi-arquivo). Leia o plano, **edite antes de aprovar** — é o momento mais barato do ciclo para corrigir uma decisão.
3. **npm run gate verde é o Done when padrão** de qualquer tarefa. Cole a saída real no **IA.md** — afirmação não é evidência.
4. **Pelo menos 1 revisão adversarial por trilha** (**/code-review** ou agente em **contexto novo** com o roteiro de **docs/code_review.md**), com triagem registrada: aceitos × recusados **com motivo**. Taxa saudável de aceitação: 30–60%. Aceitar 100% dos achados é sinal de que ninguém julgou.
5. **Higiene de contexto**: uma tarefa por sessão; regra das duas correções (errou duas vezes → **/clear** + tarefa reescrita, nunca a terceira correção na mesma sessão suja).

Nas duas rotas: o *como* é delegável. O *o quê*, o *por quê* e o **veredito** são seus — e a leitura crítica do diff continua obrigatória.

5. Os níveis de maestria

● Nível 1 — Essencial: "Refatorar sem quebrar" (trilhas ARQ + ORM)

A habilidade nomeada deste nível é a mais valiosa do tópico: **mudar a estrutura interna preservando o comportamento observável**.

Complete, na ordem:

1. **ARQ-1..3** — nasce a camada Repository: interface (*port*) + adapter SQLite para Patient, Encounter e MedicationRequest. Todo SQL sai dos services.
2. **ARQ-4 e ARQ-5** — corrija as **2 violações plantadas** da Regra da Dependência (o **npm run arch** as acusa; encontrá-las é parte do trabalho — uma delas vocês já caçaram no Encontro 1).
3. **ARQ-6** — promova a régua: a regra "só repositories importam o driver" passa a existir no **.dependency-cruiser.cjs** (a regra nasce **depois** da camada — regra que sempre grita é ruído).
4. **ORM-1..5** — Prisma entra **atrás da mesma interface** (roteiro em **prisma/LEIA-ME.md**): **init** → **db pull** → **@map/@@map** → repositories Prisma → baseline de migrations. O service não pode perceber a troca.

Pronto quando (verificável):

- `npm run gate` inteiro verde (tipos + arquitetura + testes) — a luz vermelha com que o projeto chegou apagou e **não voltou**;
- `tests/api.smoke.test.ts` **sem nenhuma alteração** e 100% verde;
- a interface visual se comporta **exatamente** como no primeiro dia;
- nenhum `import { db }` fora de `src/repositories/`.

●● Nível 2 — Proficiente: a trilha AUTH completa (inclusive o trecho sem guia)

Construa a identidade do sistema: **AUTH-1..5** (erros 401/403, tabela `users` via migration, register/login/me com `argon2` + JWT, `requireAuth`). Até aqui a Apresentação de Condução acompanha você passo a passo.

A partir daí, a apresentação para — de propósito. Você recebe apenas dois contratos: **Contrato 1 — a matriz de permissões** (também no README):

Ação	Admin	Profissional	Recepção	Sem Token
Ver pacientes/atendimentos	✓	✓	✓	401
Criar paciente / foto	✓	✓	✓	401
Registrar atendimento	✓	✓	✗ 403	401
Ver prescrições	✓	✓	✗ 403	401
Prescrever	✗ 403	✓ só no atendimento que registrou	✗ 403	401

Contrato 2 — os testes de ataque (`tests/auth.attacks.test.ts` — eles acordam sozinhos quando o login nascer — e a seção AUTH do `requests.http`).

Descobrir **o que falta construir e onde cada peça mora na arquitetura** é o coração avaliativo desta atividade. Um aviso honesto: leia a última linha da matriz com calma — há ali uma exigência que **nenhum middleware resolve sozinho**, e o lugar onde você a implementar diz mais sobre a sua compreensão de camadas do que toda a trilha ARQ.

Pronto quando (verificável):

- os **7 testes** de `auth.attacks.test.ts` acordaram e estão verdes (nenhum SKIP no relatório);
- os ataques A1–A7 do `requests.http` respondem exatamente o status anotado;
- a tela de login do frontend **acendeu sozinha** e o crachá mostra o papel;
- `npm run gate` segue inteiro verde (o N1 não pode ter regredido);
- senha em texto puro **nunca** tocou o banco, e o payload do JWT não carrega nada sensível.

●●● Nível 3 — Avançado (opcional · portfólio): prove o valor das fronteiras

Escolha **um** dos dois caminhos e escreva **uma ADR** (`docs/adr/0001-...md`, template no projeto) registrando a decisão:

- **Caminho A — Refresh token com rotação:** access token curto (15min) + refresh token persistido, rotacionado a cada uso, com revogação no logout. Honestidade técnica esperada na ADR: o que essa dupla resolve que o JWT sozinho não resolve, e o custo que ela cobra.
- **Caminho B — Segundo adapter de persistência:** `InMemory*Repository` implementando **as mesmas interfaces** da trilha ARQ, e os testes de fumaça rodando contra ele (banco nenhum no caminho). É a prova material de que os *ports* valem o que custaram. A ADR discute: quando um repositório em memória mente para você?

6. O `IA.md` — template novo (duas rotas)

Um registro **por tarefa relevante** (não por tecla digitada). Copie o bloco:

```
```markdown
Tarefa: <nome curto> · Trilha: ARQ|ORM|AUTH|N3 · Rota:
chat|agente
- **Ferramenta/modelo:**
- **Prompt (chat: C-P-T-R-F-A) ou tarefa (agente: 4 campos):**
 <cole aqui, na íntegra>
- **Plano editado?** (agente, multi-arquivo): o que você mudou no
plano e por quê
- **Evidência de pronto:** <cole a saída REAL do gate/teste que
fecha o Done when>
- **Revisão adversarial** (quando houve): achados aceitos ×
recusados, com motivo
- **O que EU decidi** (a parte que não foi delegada):
```
```

Regras: evidência é saída de terminal, não frase; pelo menos **uma recusa justificada** de achado de revisão em algum ponto da atividade; tarefa que deu errado também se registra (é dado, não vergonha).

7. Entregáveis

1. **Repositório** (link) com commits que contem a história — não um commit único "atividade pronta". Sugestão de marcos: fim de ARQ, fim de ORM, AUTH-5, matriz completa, N3.

2. **README.md atualizado por vocês:** o que foi feito por nível + a resposta à pergunta de projeto do **TODO AUTH-4** (por que o login não valida tamanho de senha com 400?).
3. **IA.md** no template da Seção 6.
4. **Autoavaliação** (meia página, no fim do README): (a) qual foi a decisão mais difícil e por quê; (b) um achado de IA que vocês **recusaram** e o motivo; (c) o que fariam diferente começando de novo.

8. Perguntas frequentes

Posso fazer a AUTH antes da ORM? Não recomendado: a tabela **users** deve nascer por migration do Prisma (invariante OP-1). Sem a baseline da ORM, você criaria a tabela por fora e teria que refazer.

O arch está verde mas eu não corriji nada — ganhei tempo? Não: você (ou sua IA) provavelmente editou a régua. Isso reprova (Seção 3, regra 4). **git diff .dependency-cruiser.cjs** revela.

Os testes de ataque não acordam. Eles sondam **POST /api/auth/login**: se ainda responde 404, dormem. Confira se o router de auth foi montado em **/api/auth** e se o **POST /login** existe.

jwt.verify lança erro e derruba tudo. Não derruba: capture e traduza para o seu **UnauthorizedError** — o **errorHandler** faz o resto. Onde essa captura mora é exatamente a pergunta da arquitetura.

Tudo responde 401 depois da AUTH. Falta **JWT_SECRET** no **.env** (copie de **.env.example**), ou vocês assinaram com um segredo e verificaram com outro.

Posso usar bcrypt em vez de argon2? Não — escopo fechado (Seção 3). A comparação entre eles foi conteúdo do Encontro 2; a atividade usa **argon2**.

A dupla dividiu: um faz ARQ, outro faz AUTH em paralelo. Má ideia — AUTH depende de ORM que depende de ARQ. Dividam por *tarefas dentro da trilha* (um conduz, outro revisa adversarialmente — e alternem).

Recepção precisa mesmo conseguir criar paciente? Sim — e há um teste que falha se vocês trancarem demais (ATAQUE 4). Autorização com mão pesada é defeito tanto quanto porta aberta.